

Report

The check that can never be true

`sqlite3_value_blob` is compiled from unmodified SQLite 3.45.1 with `clang -O3`. Along one path it tests the same flags word twice. This is the LLVM IR that clang 23.1.1 produces:

```
%3 = load i16, ptr %2          ; the flags field, 16 bits wide
%4 = zext i16 %3 to i32        ; widened by C integer promotion
%5 = and i32 %4, 18            ; guard tests the WIDE copy      (0x12)
%6 = icmp eq i32 %5, 0
br i1 %6, label %44, label %7 ; reaching %44 proves bit 1 is CLEAR

44:
%45 = and i16 %3, 514          ; query tests the NARROW original (0x202)
%46 = icmp eq i16 %45, 514     ; requires bit 1 SET
br i1 %46, label %47, label %54
```

Bit 1 is set in both masks. The guard reached block `%44` only by proving that bit is clear; the query in `%44` is taken only if that same bit is set. The branch cannot be taken, and the two basic blocks behind it cannot be reached. `-O3` keeps all of it. GCC 14, given the same file, does not. The difference is visible in the assembly of that one function and takes a moment to check:

```
gcc -O3 -std=gnu17 -S sqlite3.c -o gcc.s
clang -O3 -std=gnu17 -S sqlite3.c -o clang.s
awk '/^sqlite3_value_blob:\/,\/cfi_endproc/' clang.s | grep 514 # testl
$514, %ecx
awk '/^sqlite3_value_blob:\/,\/cfi_endproc/' gcc.s | grep 514 # nothing
```

Reducing this to a small C file does not reproduce it: clang folds the toy version. The miss depends on the surrounding function keeping several uses of the widened value alive, which is the point of the next section.

Why the compiler misses it

The two tests do not refer to the same SSA value. The guard was written about `%4`, the widened copy; the query asks about `%3`, the original.

LLVM records what a branch proves under the identity of the value the condition mentioned. In `llvm/Analysis/DomConditionCache.h` the store is

```
using AffectedValuesMap = DenseMap<Value *, SmallVector<CondBrInst *, 1>>;
```

and `conditionsFor(const Value *V)` is a lookup on that pointer. `computeKnownBitsFromContext` reaches it through `SimplifyQuery::DC`. A fact filed under `%4` is never found when something asks about `%3`, even though one is a zero-extension of the other. The information is present and correctly derived; it is filed where nobody looks for it.

Widening is not exotic. C integer promotion inserts it around every `unsigned short` or `uint8_t` field, and inlining manufactures more.

InstCombine does have a fold that narrows a masked comparison through a cast, but it is gated on the extension having exactly one use. In this function the extension has several, so the fold declines and the mismatch survives.

`tests/one-use-boundary.ll` isolates that. It holds the same function twice, the second differing only by an extra `consume(%wide)` call:

```
opt -passes=instcombine -S tests/one-use-boundary.ll
```

On stock LLVM 23.1.1 the first becomes `and i16 %flags, 18`; the second stays at `i32`. One bystander use is the entire difference, and it is what our first pass is built to handle.

What we changed

Two passes, neither of which deletes a branch or asserts that one is unreachable. Both only restate what is already there so that LLVM's own cleanup can see it.

Mask width. The identity is that zero-extension leaves the high bits clear, so testing them is redundant:

```
((zext X) & M) ==/!= C -> (X & trunc(M)) ==/!= trunc(C)
```

Applying it to one consumer of a shared extension is not a win on its own: you trade a wide `and` for a narrow one and still pay for the extension. That is why InstCombine requires a single use. We decide at the group level instead. If every consumer of an extension can be narrowed, we narrow them together and the extension dies. One consumer we cannot handle aborts the whole group, so the transform is all-or-nothing and never leaves the function worse.

Shift mask. The idiom `(1 << (x & C)) & M` tests whether the small value `x & C` is in the set encoded by `M`. A branch on that test says which values `x & C` can still take, and therefore which of its bits are now fixed. We enumerate the candidates, which is bounded and exact, and emit an `llvm.assume` naming only the bits that are fixed on that edge. If the edge turns out to be unreachable we emit nothing rather than assume `false`.

What we measured

`results/BEFORE.md` and `results/AFTER.md` are generated by the same pipeline on the same eight libraries, differing only in whether these two passes run.

Across SQLite, zlib, Lua, cJSON, bzip2, lz4, miniz and zip on LLVM 23.1.1 (3,034 functions, 44,572 branch edges), 37 branch edges are unreachable and survive `-O3`. Twelve of them need bitmask reasoning across a width cast, the combination LLVM loses; those twelve are the only ones these passes can address. Eight are removed, in six functions:

Function	Sites removed
<code>sqlite3_value_blob</code>	3
<code>trimFunc</code>	1
<code>isLikeOrGlob</code>	1
<code>jsonInsertIntoBlob</code>	1
<code>sqlite3ExprCompare</code>	1
<code>sqlite3RunVacuum</code>	1

The two passes are independent. Running each over SQLite on its own, and mining the result, gives:

Sites removed in	mask-width alone	shift-mask alone	both
<code>sqlite3_value_blob</code>	3	0	3
<code>trimFunc</code> , <code>isLikeOrGlob</code> , <code>jsonInsertIntoBlob</code> , <code>sqlite3ExprCompare</code> , <code>sqlite3RunVacuum</code>	0	5	5
Total	3	5	8

The two sets are disjoint and the totals add, so neither pass enables the other and neither is redundant. To repeat this, run `opt` over the SQLite module with one pass in the pipeline and hand the output to `bench/mine_missed_folds.py`.

Four remain. Two of them, `sqlite3VdbeMemCopy` and `sqlite3_value_dup`, no longer need a width cast to prove dead, since the cast is what our pass removed,

but the branch itself survives. We count those as not fixed rather than as a reclassification.

The other five libraries contain no instance of the pattern at all. We report that as a finding about how rare it is, not as a failure.

On the motivating function, measured on LLVM 23.1.1 with both passes:

sqlite3_value_blob	before	after
basic blocks	14	11
instructions	63	52
conditional branches	9	7
machine code	150 bytes	125 bytes

Across the whole SQLite module the passes fire 545 times. Fourteen functions shrink and seven grow, for a net change of -244 bytes. Functions grow because inlining decisions shift once a branch disappears. Both directions are reported. Compiled both ways, SQLite answers its API checks byte-identically: 15 SQL groups and 1,044 calls into the modified function.

Prior work

The bit identity is not new and we do not claim it. What we checked is whether anyone applies it where we do.

- **PR #132124**, *[ConstraintElimination] Add additional facts for bitwise AND OR* (open since March 2025). Derives sign facts from `&` and `|`. It is about bitwise operands, not width casts, and does not reach this case.
- **PR #221189**, *[InstCombine] Fold icmp (zext X), (and (trunc Y), Mask)* (open, September 2026). The closest upstream work. It compares two values rather than testing a constant mask, and it is still gated on `m_OneUse()`, the same restriction that makes our case survive.
- GCC folds the equivalent C, because it keeps equivalence classes of values rather than filing facts under one syntactic name.

So the identity is known and the one-use gate is deliberate. What we contribute is the observation that the gate is what hides a real defect class, a bounded group-level selection rule that does not need the gate, and a measurement of how often it matters.

Scope

- This is dead-code removal, not a speedup. The branches we remove were never taken, so no work disappears at run time. We measured no reliable timing difference and would not expect one.
- Machine code does not always shrink. `sqlite3_value_blob` loses 25 bytes, but in a whole-module build some functions grow as inlining decisions shift. We report both directions.
- A branch is counted only when the solver proves no input can reach it, from facts carried by dominating edges alone. Unmodelled operations become free variables and timed-out queries are discarded, so the counts are lower bounds. The timeout count moves with machine load; across five repeat runs, including one from a fresh copy of this folder, the findings did not.
- The detector is not trusted on its own. Every finding is re-evaluated by a second tool that uses no solver, walking the IR with concrete inputs to try to reach the branch. On the stock run it proved 15 dead over the entire input space, found no counterexample for 22 more in a sample of 200,000 assignments, and refuted one, in `instrFunc`, which we exclude from every count above. A sampled verdict is weaker than a proof and we do not present it as one. `results/verification-before.json` holds the verdict on each finding.
- Alive2 accepted 14 of our 16 mask-width submissions, correctly rejected a deliberately wrong mask, and returned `incomplete quantifiers` on one frozen-operand case. `results/alive2.json` records each verdict.
- The shift-mask pass has no Alive2 coverage. It emits `llvm.assume`, where a wrong derivation is a miscompile rather than a missed optimisation, so every claim it makes is instead checked against concrete arithmetic over all 65,536 inputs.
- Eight libraries is not a survey of all C code. It is the set we could compile standalone and pin by checksum.
- Building the passes emits eight deprecation warnings: LLVM 23 is mid-way through splitting `BranchInst` into `CondBrInst` and `UncondBrInst` and we still use the old spelling. Behaviour is identical. Moving to the new classes would touch validated matching code, so we did not do it without re-running every measurement.